Hands-on Exercises: Employee Management REST API using FastAPI

Objective: Build an Employee Management API using FastAPI and implement all CRUD operations using GET, POST, PUT, and DELETE.

Duration: 60 minutes Database: Not required (use a Python list) Testing: FastAPI Swagger UI — `http://127.0.0.1:8000/docs`

Exercise 1: Create the Employee Model

Create a FastAPI application named `main.py`.

Define an Employee model using Pydantic with the following fields:

Field	Data Type	Example
id	Integer	101
name	String	Rahul Sharma
department	String	IT
designation	String	Developer
salary	Float	65000
city	String	Hyderabad

Create an empty Python list named `employees` to store employee records.

Exercise 2: Add Employees (POST)

Create an endpoint:

```
POST /employees
```

Requirements:

- Accept employee information in JSON format.
- Add the employee to the list.
- Return the newly created employee.
- Add at least five employees from different departments.

Sample JSON:

```
{
  "id": 101,
  "name": "Rahul Sharma",
  "department": "IT",
  "designation": "Developer",
  "salary": 65000,
  "city": "Hyderabad"
}
```

Exercise 3: Retrieve All Employees (GET)

Create an endpoint:

`GET /employees`

Requirements:

- Return all employees.
- Verify that the five employees added earlier are displayed.
- Check the response when the list is empty.

Exercise 4: Retrieve Employee by ID (GET)

Create an endpoint:

`GET /employees/{employee_id}`

Requirements:

- Accept an employee ID as a path parameter.
- Search for the employee.
- Return the matching employee.
- Return HTTP 404 if the employee does not exist.

Test with employee IDs `101`, `102`, and `999`.

Exercise 5: Update Employee (PUT)

Create an endpoint:

```
PUT /employees/{employee_id}
```

Requirements:

- Accept an employee ID.
- Accept updated employee information in JSON format.
- Find and update the employee.
- Return the updated record.
- Return HTTP 404 if the employee does not exist.

Tasks:

- Change employee 101's salary from ₹65,000 to ₹80,000.
- Change employee 102's department to HR.
- Change employee 103's city to Bengaluru.

Verify the updates using GET requests.

Exercise 6: Delete Employee (DELETE)

Create an endpoint:

```
DELETE /employees/{employee_id}
```

Requirements:

- Accept an employee ID.
- Remove the employee from the list.
- Return a success message.
- Return HTTP 404 if the employee does not exist.

Tasks:

- Delete employee 102.
- Retrieve all employees to verify deletion.
- Try deleting employee 102 again.
- Observe the HTTP response.